

✓ Subword Embedding

:label: `sec_fasttext`

In English, words such as "helps", "helped", and "helping" are inflected forms of the same word "help". The relationship between "dog" and "dogs" is the same as that between "cat" and "cats", and the relationship between "boy" and "boyfriend" is the same as that between "girl" and "girlfriend". In other languages such as French and Spanish, many verbs have over 40 inflected forms, while in Finnish, a noun may have up to 15 cases. In linguistics, morphology studies word formation and word relationships. However, the internal structure of words was neither explored in word2vec nor in GloVe.

The fastText Model

Recall how words are represented in word2vec. In both the skip-gram model and the continuous bag-of-words model, different inflected forms of the same word are directly represented by different vectors without shared parameters. To use morphological information, the *fastText* model proposed a *subword embedding* approach, where a subword is a character n -gram

:cite: `Bojanowski.Grave.Joulin.ea.2017`. Instead of learning word-level vector representations, fastText can be considered as the subword-level skip-gram, where each *center word* is represented by the sum of its subword vectors.

Let's illustrate how to obtain subwords for each center word in fastText using the word "where". First, add special characters "<" and ">" at the beginning and end of the word to distinguish prefixes and suffixes from other subwords. Then, extract character n -grams from the word. For example, when $n = 3$, we obtain all subwords of length 3: "<wh", "whe", "her", "ere", "re>", and the special subword "<where>".

In fastText, for any word w , denote by $\mathcal{G}_w$ the union of all its subwords of length between 3 and 6 and its special subword. The vocabulary is the union of the subwords of all words. Letting $\mathbf{z}_g$ be the vector of subword g in the dictionary, the vector $\mathbf{v}_w$ for word w as a center word in the skip-gram model is the sum of its subword vectors:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g.$$

The rest of fastText is the same as the skip-gram model. Compared with the skip-gram model, the vocabulary in fastText is larger, resulting in more model parameters. Besides, to calculate the representation of a word, all its subword vectors have to be summed, leading to higher computational complexity. However, thanks to shared

parameters from subwords among words with similar structures, rare words and even out-of-vocabulary words may obtain better vector representations in fastText.

Byte Pair Encoding

:label: `subsec_Byte_Pair_Encoding`

In fastText, all the extracted subwords have to be of the specified lengths, such as 3 to 6, thus the vocabulary size cannot be predefined. To allow for variable-length subwords in a fixed-size vocabulary, we can apply a compression algorithm called *byte pair encoding* (BPE) to extract subwords :cite: `Sennrich.Haddow.Birch.2015`.

Byte pair encoding performs a statistical analysis of the training dataset to discover common symbols within a word, such as consecutive characters of arbitrary length. Starting from symbols of length 1, byte pair encoding iteratively merges the most frequent pair of consecutive symbols to produce new longer symbols. Note that for efficiency, pairs crossing word boundaries are not considered. In the end, we can use such symbols as subwords to segment words. Byte pair encoding and its variants has been used for input representations in popular natural language processing pretraining models such as GPT-2 :cite: `Radford.Wu.Child.ea.2019` and RoBERTa :cite: `Liu.Ott.Goyal.ea.2019`. In the following, we will illustrate how byte pair encoding works.

First, we initialize the vocabulary of symbols as all the English lowercase characters, a special end-of-word symbol `'_'`, and a special unknown symbol `'[UNK]'`.

```
import collections

symbols = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
           'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z',
           '_', '[UNK]']
```

Since we do not consider symbol pairs that cross boundaries of words, we only need a dictionary `raw_token_freqs` that maps words to their frequencies (number of occurrences) in a dataset. Note that the special symbol `'_'` is appended to each word so that we can easily recover a word sequence (e.g., "a taller man") from a sequence of output symbols (e.g., "a_ tall er_ man"). Since we start the merging process from a vocabulary of only single characters and special symbols, space is inserted between every pair of consecutive characters within each word (keys of the dictionary `token_freqs`). In other words, space is the delimiter between symbols within a word.

```
raw_token_freqs = {'fast_': 4, 'faster_': 3, 'tall_': 5, 'taller_': 4}
token_freqs = {}
```

```
for token, freq in raw_token_freqs.items():
    token_freqs[' '.join(list(token))] = raw_token_freqs[token]
token_freqs

{'f a s t _': 4, 'f a s t e r _': 3, 't a l l _': 5, 't a l l e r _': 4}
```

We define the following `get_max_freq_pair` function that returns the most frequent pair of consecutive symbols within a word, where words come from keys of the input dictionary `token_freqs`.

```
def get_max_freq_pair(token_freqs):
    pairs = collections.defaultdict(int)
    for token, freq in token_freqs.items():
        symbols = token.split()
        for i in range(len(symbols) - 1):
            # Key of `pairs` is a tuple of two consecutive symbols
            pairs[symbols[i], symbols[i + 1]] += freq
    return max(pairs, key=pairs.get) # Key of `pairs` with the max value
```

As a greedy approach based on frequency of consecutive symbols, byte pair encoding will use the following `merge_symbols` function to merge the most frequent pair of consecutive symbols to produce new symbols.

```
def merge_symbols(max_freq_pair, token_freqs, symbols):
    symbols.append(' '.join(max_freq_pair))
    new_token_freqs = dict()
    for token, freq in token_freqs.items():
        new_token = token.replace(' '.join(max_freq_pair),
                                   ' '.join(max_freq_pair))
        new_token_freqs[new_token] = token_freqs[token]
    return new_token_freqs
```

Now we iteratively perform the byte pair encoding algorithm over the keys of the dictionary `token_freqs`. In the first iteration, the most frequent pair of consecutive symbols are `'t'` and `'a'`, thus byte pair encoding merges them to produce a new symbol `'ta'`. In the second iteration, byte pair encoding continues to merge `'ta'` and `'l'` to result in another new symbol `'tal'`.

```
num_merges = 10
for i in range(num_merges):
    max_freq_pair = get_max_freq_pair(token_freqs)
    token_freqs = merge_symbols(max_freq_pair, token_freqs, symbols)
    print(f'merge #{i + 1}:', max_freq_pair)

merge #1: ('t', 'a')
merge #2: ('ta', 'l')
merge #3: ('tal', 'l')
merge #4: ('f', 'a')
```

```
merge #5: ('fa', 's')
merge #6: ('fas', 't')
merge #7: ('e', 'r')
merge #8: ('er', '_')
merge #9: ('tall', '_')
merge #10: ('fast', '_')
```

After 10 iterations of byte pair encoding, we can see that list `symbols` now contains 10 more symbols that are iteratively merged from other symbols.

```
print(symbols)
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', ' '
```

For the same dataset specified in the keys of the dictionary `raw_token_freqs`, each word in the dataset is now segmented by subwords "fast_", "fast", "er_", "tall_", and "tall" as a result of the byte pair encoding algorithm. For instance, words "faster_" and "taller_" are segmented as "fast er_" and "tall er_", respectively.

```
print(list(token_freqs.keys()))
```

```
['fast_', 'fast er_', 'tall_', 'tall er_']
```

Note that the result of byte pair encoding depends on the dataset being used. We can also use the subwords learned from one dataset to segment words of another dataset. As a greedy approach, the following `segment_BPE` function tries to break words into the longest possible subwords from the input argument `symbols`.

```
def segment_BPE(tokens, symbols):
    outputs = []
    for token in tokens:
        start, end = 0, len(token)
        cur_output = []
        # Segment token with the longest possible subwords from symbols
        while start < len(token) and start < end:
            if token[start: end] in symbols:
                cur_output.append(token[start: end])
                start = end
                end = len(token)
            else:
                end -= 1
        if start < len(token):
            cur_output.append('[UNK]')
        outputs.append(' '.join(cur_output))
    return outputs
```

In the following, we use the subwords in list `symbols`, which is learned from the aforementioned dataset, to segment `tokens` that represent another dataset.

```
tokens = ['tallest_', 'fatter_']  
print(segment_BPE(tokens, symbols))
```

```
['tall e s t _', 'fa t t er_']
```

✓ Summary

- The fastText model proposes a subword embedding approach. Based on the skip-gram model in word2vec, it represents a center word as the sum of its subword vectors.
- Byte pair encoding performs a statistical analysis of the training dataset to discover common symbols within a word. As a greedy approach, byte pair encoding iteratively merges the most frequent pair of consecutive symbols.
- Subword embedding may improve the quality of representations of rare words and out-of-dictionary words.

Exercises

1. As an example, there are about 3×10^8 possible 6-grams in English. What is the issue when there are too many subwords? How to address the issue? Hint: refer to the end of Section 3.2 of the fastText paper :cite:Bojanowski.Grave.Joulin.ea.2017).
2. How to design a subword embedding model based on the continuous bag-of-words model?
3. To get a vocabulary of size m , how many merging operations are needed when the initial symbol vocabulary size is n ?
4. How to extend the idea of byte pair encoding to extract phrases?

[Discussions](#)

